# statistics.py — technical

`src/moe_pruning/scoring/statistics.py · 175 lines · moe_pruning.scoring.statistics`

## Purpose

Running float64 sums per layer and expert.

## Data and structure

- Per expert: count, gate_sum, norm_sum, contribution_sum, importance, output_sum[D], output_sq_sum; bias split: bias_count, w_sum, w_output_sum[D], w_output_sq_sum, g2_sum, g2_output_sum[D], g2_output_sq_sum.
- Per layer: tokens, dropped_tokens, contribution_sum_all.

## Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

### `class LayerStatistics`

lines 57-88 · dataclass

Float64 running sums of one layer, per expert and per layer.

| Field | Type | Default |
|---|---|---|
| `layer_id` | str | |
| `num_experts` | int | |
| `dim` | int | |
| `expert` | dict[str, torch.Tensor] | field(default_factory=dict) |
| `layer` | dict[str, float] | field(default_factory=dict) |

#### `LayerStatistics.observed`

```
def observed(self) -> torch.Tensor
```

lines 74-75

Which experts received at least one real token.

#### `LayerStatistics.to_state`

```
def to_state(self) -> dict
```

lines 77-84

Dictionary for torch.save.

#### `LayerStatistics.from_state`

```
def from_state(cls, state: dict) -> 'LayerStatistics'
```

lines 87-88

Restore from the saved dictionary.

### `class ExpertStatisticsAccumulator`

lines 91-171

Collects LayerStatistics for many layers from forward records.

#### `ExpertStatisticsAccumulator.update`

```
def update(self, record: MoEForwardRecord, token_mask: torch.Tensor, bias_mask: torch.Tensor |
None=None) -> None
```

lines 99-168

Add one forward record.

1. Flatten masks; `for_bias` = bias mask restricted to real tokens.
2. On the model device: s_t for real tokens; then move index `[N,K]`, gate, expert outputs `[N,K,D]` and s to CPU.

3. Layer counters: real tokens, tokens with no expert in any slot, sum of s.
4. For each slot k: tokens routed in that slot; float64 gate g, output E, s; `index_add_` count, sums of g, ||E||, s, g*||E||*s (importance), E and ||E||^2 into their expert rows.
5. For tokens in the bias split additionally add w = s*g^2 and g^2 weighted sums (scalars, vectors, squared norms).

### `ExpertStatisticsAccumulator.state`

```
def state(self) -> dict
```

lines 170-171

Saved form of all layers' statistics.

### `load_statistics`

```
def load_statistics(state: dict) -> dict[str, LayerStatistics]
```

lines 174-175

Restore LayerStatistics objects from expert_stats.pt.

## Module constants

| Name | Value |
|---|---|
| EXPERT_SCALARS | ('count', 'gate_sum', 'norm_sum', 'contribution_sum', 'importance', 'output_sq_sum', 'bias_count', 'w_sum', 'w... |
| EXPERT_VECTORS | ('output_sum', 'w_output_sum', 'g2_output_sum') |
| LAYER_SCALARS | ('tokens', 'dropped_tokens', 'contribution_sum_all') |